

★ Member-only story

I Used Karpathy's LLM Wiki to Build a Research Brain That Updates Itself. Here's What Two Weeks Taught Me.

Here is a situation every developer who reads a lot will recognise.

11 min read · Apr 19, 2026



Adi Insights and Innovations

Follow



Listen



Share

... More

You have a folder somewhere called “research” or “reading” or “ai-stuff.” Inside it are forty-seven markdown files, six PDFs you meant to summarise, three Notion pages you exported once and never opened again, and a browser bookmark folder called “important” that contains links from 2023. You know there is something in there about attention mechanisms and something else about MCP servers, and you are almost certain those two things are related, but you have no idea where you wrote that down or whether you wrote it down at all.



You are not disorganised. You are overloaded. There is a difference.

I found a solution. It involves a one-page idea document by Andrej Karpathy, an AI coding agent called Claude Code, and a free note-taking app called Obsidian. In about two hours — not thirty minutes, I will be honest about what it actually took — I went from a folder of disconnected documents to a living, cross-referenced knowledge base that maintains itself. And in two weeks of daily use, it has already surfaced connections I would never have made manually.

I did not write a single wiki page myself.

. . .

The problem with RAG and why I kept hitting its ceiling

Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but the LLM is rediscovering knowledge from scratch on every question. There is no accumulation. Ask a subtle question that requires synthesising five documents, and the LLM has to find and piece together the relevant fragments every time. Nothing is built up.

I had been using this pattern for months. NotebookLM for papers. ChatGPT file uploads for meeting notes. A collection of Claude conversations I would never look at again. Each one was a one-shot transaction. I would ask a question, get an answer, close the tab, and two weeks later I would be re-uploading the same papers to ask almost the same question, because I had not retained the synthesis anywhere durable.

The hardest part of building a knowledge base is not reading information. It is maintaining it. Humans are not great at this at scale. It becomes messy very quickly. LLMs, on the other hand, are very good at exactly this kind of work.

That observation stopped me. I had been treating the maintenance problem as a human problem, which meant it was a willpower problem, which meant it was a problem I would perpetually lose. Karpathy's insight is that it does not have to be a human problem at all.

What Karpathy's LLM Wiki actually is

In April 2026, Andrej Karpathy caused a stir in the AI community when he dropped a GitHub Gist titled simply "LLM Wiki." It was not a new app, a library, or a SaaS product. It was an "idea file" — a conceptual pattern designed to be copy-pasted into an LLM agent like Claude Code or OpenAI Codex. He described a pattern he called an "LLM Wiki." The idea is simple but the shift is significant: instead of you maintaining a knowledge base and occasionally asking AI questions about it, the LLM builds and maintains the entire knowledge base for you. You collect raw sources — articles, papers, book notes, podcast takeaways, anything — you drop them into a folder, and then you tell the LLM to "compile" them into a wiki. It reads every source, writes summary pages, creates pages for key people and concepts, and cross-references everything. The analogy Karpathy reaches for is from software engineering: compilation. When you write source code, a compiler transforms it into an optimised binary artefact. You do not execute the source code directly every time — you compile once, distribute the artefact, and run it efficiently on demand. The compilation step is expensive, but it pays for itself across every subsequent use.

The compiled artefact, in this case, is not a binary. It is a wiki — a folder of human-readable markdown files, one per concept, with cross-references between them, maintained by the model, viewable in Obsidian.

RAG retrieves and forgets. A wiki accumulates and compounds.

That is the entire bet.

The three-layer architecture: what goes where and why

The pattern has three layers: raw sources (immutable documents — PDFs, transcripts, bookmarks, notes), the wiki (LLM-generated markdown — summaries, entity pages, cross-references, contradiction flags), and the schema (a CLAUDE.md file that tells the LLM how to maintain the wiki). The raw sources are your inputs. The wiki is the LLM's persistent, evolving understanding of those inputs. The schema is the operating manual.

The immutability of the raw layer is not an aesthetic choice. It is a deliberate design decision. It means you can always re-derive the wiki from scratch if needed. Your raw sources are the ground truth. The wiki is the interpretation. If the interpretation ever goes wrong — if the LLM makes a bad inference during ingest and it propagates — you can lint it, fix it, or rebuild from scratch. You never lose the originals.

The schema document — CLAUDE.md — is the most important file in the system. It is what turns a generic LLM into a disciplined knowledge worker. Mine took about forty minutes to write with Claude's help. It defines what page types exist in my wiki (entities, concepts, sources, syntheses), the naming conventions for `[[wikilinks]]`, when to create a new page versus update an existing one, and what the lint checklist should check. The first version will be rough. After a few dozen sources and a few lint passes, you will have a schema that reflects how your domain actually works.

The build: two hours, not thirty minutes

I set up the folder structure first. A `raw/` directory for sources. A `wiki/` directory for compiled pages. The CLAUDE.md schema in the root. An `index.md` and a `log.md` to help the model navigate as the wiki grows.

```
my-wiki/
├── raw/
│   ├── papers/
│   ├── articles/
│   └── notes/
├── wiki/
│   ├── entities/
│   ├── concepts/
│   ├── sources/
│   └── index.md
```

Then I opened Claude Code, pointed it at the directory, and gave it the gist. The setup took about fifteen minutes. The first ingest — dropping in a paper I had been meaning to read and asking Claude to process it — took another thirty. That first pass was where I understood what the system was actually doing.

When you drop a new source into the raw collection and tell the LLM to process it, the LLM reads the source, discusses key takeaways with you, writes a summary page in the wiki, updates the index, updates relevant entity and concept pages across the wiki, and appends an entry to the log. A single source might touch 10–15 wiki pages. Karpathy prefers to ingest sources one at a time and stay involved — reading the summaries, checking the updates, and guiding the LLM on what to emphasise.

That last point matters. The first ingest is a conversation, not a batch job. Claude summarised the paper, proposed five concept pages it wanted to create, and asked whether I wanted a page for the specific dataset used or just the methodology. Those are editorial decisions. The model was right to ask.

The remaining hour was spent doing four more ingests, watching the wiki grow, and refining the schema when I noticed Claude was creating slightly inconsistent page structures between runs.

The three operations that keep it alive

The three-layer architecture — raw sources (you curate, immutable), the wiki (the LLM writes, you read), and the schema file (CLAUDE.md) — drives the system. Three operations run everything: Ingest (one source touches 10 to 15 wiki pages), Query (answers get filed as new pages), and Lint (weekly health check that fixes contradictions and orphan pages).

Ingest is the operation you run most often. Drop a source in `raw/`, tell Claude to ingest it, stay involved while it works. After two weeks, I have ingested 34 sources — papers, articles, YouTube transcripts saved as markdown, podcast notes — and the wiki has grown to 87 pages without me writing a single one.

Query is where the compounding pays off. The quality difference from traditional RAG is significant. The LLM is not reading a random chunk from page 14 of a PDF. It is reading a pre-synthesised, cross-referenced encyclopaedia entry that already integrates everything the system has ever learned about that concept from every source ever ingested. When I asked “what does everything I’ve read say about the relationship between context window length and agent reliability,” Claude read the index, identified six relevant wiki pages, and produced a synthesis that referenced four different papers by name and flagged one contradiction between them.

There is one more step that completes the loop: the `— save` flag. When enabled, a synthesised answer that represents new, valuable knowledge is automatically filed back as a new wiki page. The slug is derived from the question itself. Future sessions benefit immediately. This is what Karpathy calls the compounding principle. You asked a question, the system answered it, and now the wiki knows the answer too.

Lint is the operation nobody talks about and everyone needs. Periodically, you ask the LLM to health-check the wiki. It looks for: contradictions between pages, stale claims that newer sources have superseded, orphan pages with no inbound links, important concepts mentioned but lacking their own page, missing cross-references, data gaps that could be filled with a web search. The LLM is good at suggesting new questions to investigate and new sources to look for. This keeps the wiki healthy as it grows.

I ran my first lint pass after twelve days. Claude flagged three contradictions between papers I had ingested in week one and week two — one of them a real disagreement in the literature that I had completely missed. It flagged two orphan pages. It suggested four concepts that were referenced across multiple pages but had no page of their own. The whole pass took about eight minutes.

Flagging contradictions during a lint pass takes the LLM three minutes. The intellectual overhead that caused every human-maintained wiki to eventually decay is gone. What’s left is the part that’s genuinely human: curating sources, deciding what matters, asking the right questions, and thinking about what it all means. The LLM handles everything else.

What Obsidian is actually doing

Obsidian is not the brain here. Claude Code is the brain. Obsidian is the window.

Obsidian acts as the IDE for the wiki. It renders markdown with backlinks, shows you a graph of how pages connect, lets you search across all pages, and updates in real time as files change on disk. You are always in reading mode, never in editing mode. You browse, follow links, check the graph, and read. You never directly edit wiki files.

The graph view is the feature that converts sceptics. After two weeks and 87 pages, the graph shows natural clusters — one around transformer architecture, one around agent memory, one around the evaluation literature — with specific pages acting as bridges between clusters. I did not design those clusters. The model found them by following what the literature actually discussed together.

As community member Charly Wagnier observed, “It acts as a living AI knowledge base that actually heals itself.” The graph view is where you see that in practice — the connections are not things you designed, they are things the literature actually contains.

• • •

What actually breaks (and how to fix it)

Two weeks in, here is what nobody’s tutorial mentions.

[Open in app](#) ↗

≡ **Medium**



“Main Arguments,” and they mean the same thing. Fix the schema. Standardise the section names. The model will follow whatever the schema says, which means the schema needs to say the right thing clearly. The schema feels overengineered at first. With 10 pages, defining 5 page types and 8 lint rules seems like overkill. Past 50 pages, you will be grateful for the consistency.

Epistemic drift is real. If the LLM misinterprets a source during ingest — a misinterpretation, a hallucinated connection — that error becomes part of the wiki and can influence future ingests. The lint operation mitigates this but does not eliminate it. Human review of wiki edits via git diffs is the real safety net. I run `git diff` after every ingest. It takes ninety seconds and has caught two errors in two weeks.

It hits a wall around 100–200 sources. Karpathy advises a manageable threshold, suggesting wikis contain around 100 articles to ensure the system remains efficient. Beyond that, the pattern works best for personal or team knowledge at the 50–200 source scale. Beyond that, you likely need the extensions from LLM Wiki v2 — hybrid search, multi-agent governance — or a proper RAG pipeline. The good news is that 200 well-curated sources, properly compiled and cross-referenced, is more usable than 2,000 raw documents in a RAG system that rediscovers them from scratch every query.

The ingest cost is front-loaded. The ingest step is where most token cost concentrates. A 10-page PDF might trigger updates across 12 wiki files. Budget roughly 5–8× the source token count for a full ingest pass. Subsequent queries against the compiled wiki are far cheaper — typically just index.md plus 2–3 article pages. I spend maybe 15 minutes and a modest Claude API budget per ingest. The queries after that are nearly free by comparison.

. . .

What changed after two weeks

I stopped losing things.

That sounds trivial. It is not. Before this system, every interesting paper I read existed in a state of gradual decay — it was most useful the day I read it and progressively less accessible every week after that. Two weeks in, the opposite is happening. Each new source makes the existing ones more findable, because the ingestion process explicitly updates every related page. The paper I read on day one is now cross-referenced on eleven wiki pages I added later. It is more connected than when I first read it.

Karpathy rarely touches the wiki files himself. He treats the wiki as “AI territory.” The AI writes it, updates it, and maintains it. Karpathy just reads it.

That is the discipline that makes the system work. The moment you start manually editing wiki pages, you are competing with the model on its own terrain and losing the maintenance automation that makes the whole thing viable. Read the wiki. Ask questions. Feed it sources. Let it do the filing.

Karpathy mentions a future direction: using the wiki to generate synthetic training data and fine-tune an LLM so it “knows” the data in its weights rather than just through context windows. That would turn a personal knowledge base into a personalised model.

That future is further away than Karpathy’s gist makes it sound. But the present version — a self-healing, cross-referenced, LLM-maintained wiki that runs in a folder on your local machine — is already the best knowledge management system I have ever used.

The Memex is finally buildable. Not because we have better documents or better search, but because we have librarians that actually do the work.

Vannevar Bush described it in 1945. Karpathy published the instructions in April 2026.

The folder is right there. The gist is public. Claude Code has a free tier. The only thing left is the first ingest.

LLM

Llm Applications

AI

Furture Of Work

Wiki



Following

Published in Towards AI

134K followers · Last published just now

We build Enterprise AI. We teach what we learn. Join 100K+ AI practitioners on Towards AI Academy. Free: 6-day Agentic AI Engineering Email Guide: <https://email-course.towardsai.net/>



Follow



Written by Adi Insights and Innovations

580 followers · 78 following

Tech enthusiast, AI explorer, and innovator. Writing to inspire and unlock the potential of cutting-edge technologies. Join me on a journey of discovery!

Responses (2)



Witek ten Hove

What are your thoughts?

See all responses